

QUÓRUM

La verdad tiene quórum.

Pipeline de dos modelos en Android

Portero 0.8B · Extractor 1.7B · Verificador determinista

Análisis crítico, código y prompt maestro

Reto corporativo de Philips · Decentralized AI Hackathon

Anexo D del documento maestro

QUÓRUM · Pipeline de dos modelos en Android

Análisis crítico, arquitectura corregida y prompt maestro de construcción

La verdad tiene quórum. Anexo D del documento maestro. Cubre el pipeline Portero + Extractor, su implementación en Android con Expo, y el prompt listo para pegar a un agente de codificación.

PARTE 1 · ANÁLISIS CRÍTICO DEL DISEÑO PROPUESTO

1.1 Lo que está bien y hay que conservar

El diseño de dos modelos que se vigilan tiene tres aciertos que no son obvios y que la mayoría de los equipos no va a tener:

Acierto 1 — "nadie borra, solo el acuerdo escribe". Es el instinto correcto. Un filtro que descarta silenciosamente es peor que no tener filtro, porque el usuario nunca se entera de lo que perdió. Marcar en vez de borrar convierte un falso negativo catastrófico en una molestia menor.

Acierto 2 — la métrica que eligieron. Decir que "la métrica que decide no es el acierto del portero sino la tasa de interrupción" es una observación de producto, no de ingeniería, y es correcta. Un sistema que pregunta demasiado no se usa dos veces. Muy poca gente elige la métrica de fricción por encima de la métrica de exactitud.

Acierto 3 — la tercera fila. Convertir la alucinación en una pregunta en lugar de en una fila falsa es exactamente lo que hace falta. Una fila falsa contamina el dataset para siempre; una pregunta cuesta cinco segundos.

Esos tres los conservo íntegros.

1.2 Problema grave nº1 — las cifras de 93 % y 95 % no pueden usarse

"El extractor inventa equipo en el 93 % de las notas donde no hay nada; el portero acierta en el 95 % de esas mismas notas."

Si esas cifras salieron de una corrida real sobre un conjunto etiquetado, son valiosas y hay que documentar la metodología: cuántas notas, cómo se etiquetaron, qué modelo exacto, qué cuantización, qué temperatura, qué prompt. Si no salieron de una corrida real, **no pueden aparecer en el README ni en el video.**

El texto de la tercera imagen dice "Pendiente de medir sobre las 300 notas ciegas" — o sea que la medición **todavía no se hizo**. Entonces esos dos porcentajes son estimaciones, y presentarlos como resultados es exactamente el tipo de cosa que un jurado técnico verifica y que hunde la credibilidad de todo lo demás, incluido lo que sí funciona.

Regla: o se mide y se reporta con metodología, o se dice "esperamos que" y se marca como hipótesis. No hay término medio.

1.3 Problema grave nº2 — "no comparten la ceguera" es una suposición, no un hecho

Esta es la falla de diseño más importante y la razón por la que hay que corregir la arquitectura.

La tesis del diseño es que dos modelos fallan de forma **independiente**, y por eso cruzarlos detecta errores. Pero un Qwen3.5 de 0.8B y un Qwen3 de 1.7B:

- vienen de la **misma familia** y muy probablemente del mismo linaje de preentrenamiento,
- comparten tokenizador y estilo de ajuste por instrucciones,
- están cuantizados con el mismo esquema,
- y reciben **el mismo texto de entrada**.

Cuando una nota es ambigua —"pasé por el hospital, había equipo viejo por todos lados"— hay razones de sobra para pensar que **ambos se equivocan en la misma dirección**. Sus errores no son independientes: están correlacionados.

Esto no es una objeción teórica. Es el hallazgo H-04 del blueprint FOCUS: "verificación correlacionada con generación → el verificador debe ser un modelo o una heurística distinta". Aquí el verificador es un primo hermano del generador.

Lo que hay que hacer: no se puede *asumir* la independencia, hay que **medirla**. La métrica que importa no es "el portero acierta 95 %", sino la **tasa de fallo conjunto**: en qué porcentaje de casos ambos se equivocan a la vez. Si el fallo conjunto es alto, el segundo modelo está gastando batería sin aportar seguridad.

Y mientras tanto, hay que añadir un verificador que **sí** sea independiente por construcción. Que es lo siguiente.

1.4 La corrección que cambia el diseño — evidencia citada, verificada por código

Aquí está mi aporte principal, y es lo que convierte un diseño frágil en uno sólido.

Obligamos al extractor a citar. Cada fila que produce debe venir acompañada del **fragmento literal de la nota** que la justifica. No un resumen: el texto exacto.

```
lote: { modalidad: "MR", cantidad: 2, marca: "NovaMed" }
evidencia: "tienen dos resonadores NovaMed" - literal, de la nota
```

Y entonces el código —sin ningún modelo, sin latencia, sin batería— hace una comprobación trivial:

```
notaNormalizada.includes(normalizar(evidencia))
```

Si el fragmento citado **no aparece en la nota**, la fila está fabricada. Punto. No hay opinión, no hay umbral, no hay correlación de errores: es una comparación de cadenas.

Por qué esto es superior al portero:

	Portero (0.8B)	Evidencia citada (código)
Independencia del extractor	Dudosa: misma familia, mismo input	Total: no es un modelo
Costo	Un modelo más en RAM, latencia, batería	Cero
Determinismo	No	Sí
Auditable ante el jurado	"confía en el 95 %"	Se ve la cadena y se compara
Detecta alucinación de fila entera	A veces	Siempre

Esto no elimina al portero, lo reubica. El portero sigue sirviendo para algo que la evidencia citada no puede hacer: detectar **omisiones** (la nota sí menciona equipo pero el extractor no sacó nada). Ahí un segundo modelo aporta. Para las alucinaciones, el código es mejor.

1.5 Problema nº3 — el marco conceptual choca con la tesis de QUÓRUM

El diseño dice "*solo el acuerdo escribe*". Suena bien, pero en QUÓRUM **ya existe una compuerta antes de escribir: el humano** (hueco H-03, paso 12 de la lógica de Philips). Nada se persiste sin confirmación.

Si presentamos el cruce de dos modelos como "lo que decide qué se guarda", estamos diciendo que **dos LLM deciden la verdad** — que es lo contrario exacto de nuestra tesis, la que repetimos en el video: *el modelo entiende, el sistema decide*.

El reencuadre correcto, y es solo una frase:

El cruce Portero × Extractor **no decide qué es verdad. Decide si el usuario ve un borrador limpio o una pregunta.** Es un **enrutador de interrupciones**, no un árbitro de verdad. El árbitro sigue siendo el humano, y después el quórum entre observadores.

Con ese reencuadre, el pipeline encaja perfecto en la arquitectura y refuerza la tesis en vez de contradecirla. Sin él, un jurado atento nos pregunta "*¿entonces el modelo sí decide?*" y no tenemos respuesta.

1.6 Problema nº4 — la transcripción no la hace Qwen

Corrección técnica directa: **Qwen 3 1.7B no transcribe audio.** Es un modelo de lenguaje, no un modelo de voz.

La transcripción en QVAC la hace el addón `transcription-whispercpp`, que está marcado como *stable* en el catálogo oficial. La cadena real es:

```
audio → whisper.cpp (ASR) → texto → Portero 0.8B → Extractor 1.7B
```

Son **tres modelos**, no dos. Eso importa para el presupuesto de memoria del teléfono, que es lo que decide si esto corre o no.

1.7 Problema nº5 — no hay presupuesto de interrupciones

El diseño dice que en dos de los cuatro casos "se pregunta". No dice **cuántas veces**. Sin un tope:

- Una nota confusa puede generar una pregunta por cada lote dudoso.
- Si el usuario no responde, no está definido qué pasa con la nota.
- Un peer malicioso podría inyectar notas diseñadas para maximizar preguntas: **denegación de servicio por fricción**.

Regla que hay que añadir: máximo **una** pregunta por nota. Si el usuario la ignora o la pospone, la nota se guarda igual con estado `pendiente-de-revision` y **nunca se descarta**. La cola de pendientes es visible en la interfaz.

1.8 Problema nº6 — el orden de ejecución desperdicia el modelo barato

En el diagrama, Portero y Extractor corren en paralelo desde la nota. Eso obliga a cargar y ejecutar el modelo de 1.7B **siempre**, incluso en las notas donde no hay nada.

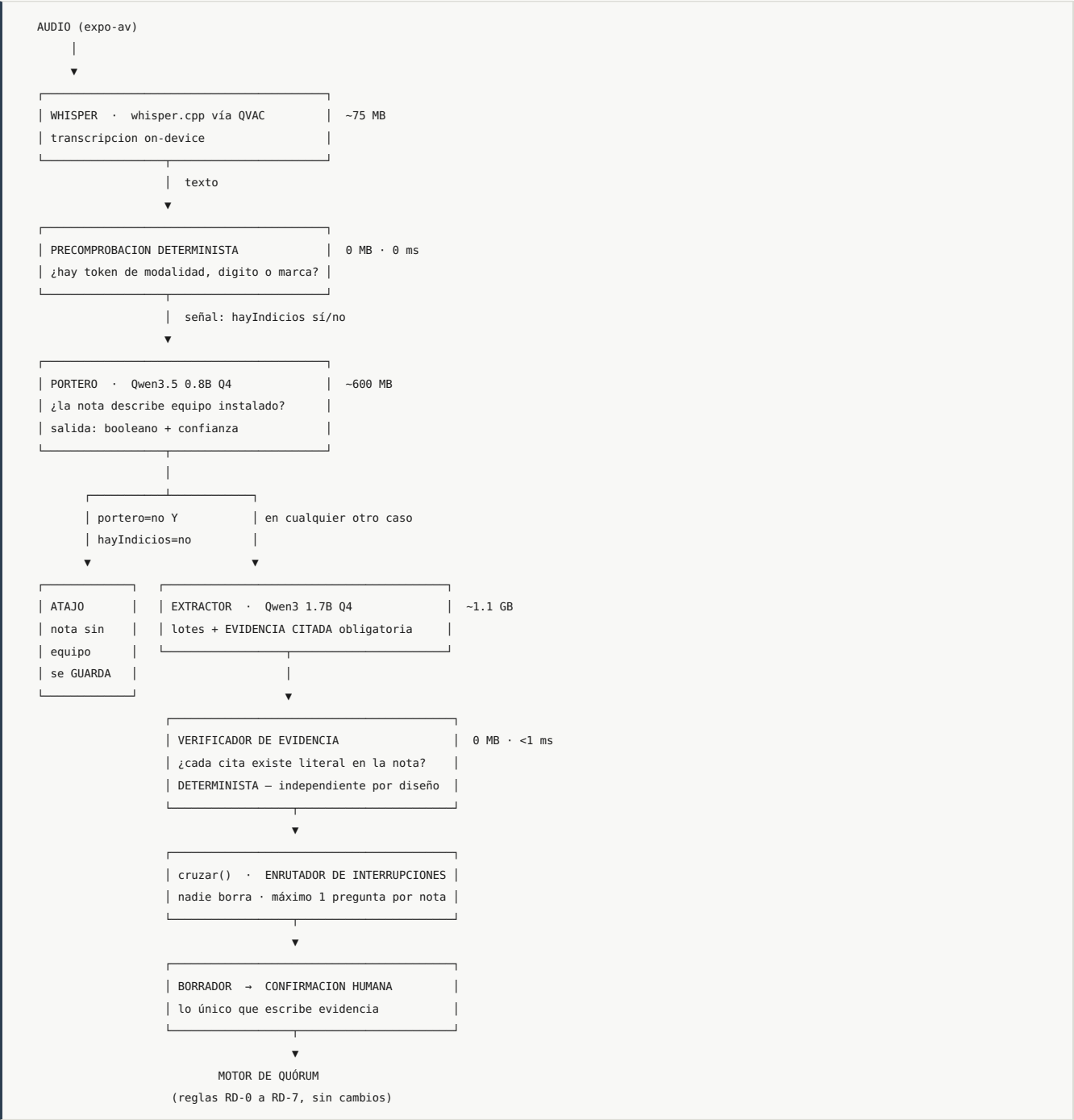
Pero el cruce necesita ambas salidas para funcionar... salvo en un caso: cuando el portero dice **no** y una **precomprobación determinista** también dice que no hay nada. Si el texto normalizado no contiene ninguna palabra de modalidad, ningún dígito y ninguna marca conocida, y además el portero dice no, hay dos señales independientes coincidiendo — y una de ellas no es un modelo.

En ese caso concreto se puede saltar el extractor. En un teléfono, saltarse el modelo grande es la diferencia entre 1,2 segundos y 6 segundos, y entre gastar batería o no.

Lo importante: ese atajo **nunca descarta la nota**. La guarda como testimonio sin equipo, que es un dato válido y útil ("*visité este cliente y no observé equipo*" es información).

PARTE 2 · ARQUITECTURA CORREGIDA

2.1 La cadena completa



2.2 La tabla de decisión corregida

Cinco resultados, no cuatro, y el nuevo es el más importante:

Portero	Lotes	Evidencia	Resultado	Qué ve el usuario
sí	N>0	toda válida	ACUERDO	Borrador limpio para confirmar
no	0	—	ACUERDO_VACIO	"Nota guardada, sin equipo detectado"
no	N>0	toda válida	POSIBLE_OMISION_PORTERO	"Detecté esto, ¿es correcto?"
sí	0	—	POSIBLE_OMISION_EXTRACTOR	"Creo que mencionaste equipo, ¿puedes repetirlo?"
—	N>0	alguna inválida	EVIDENCIA_FABRICADA	Las filas sin respaldo se marcan , no se guardan; las válidas siguen

La última fila es la que aporta el verificador determinista, y es la única que no depende de que dos modelos no compartan la ceguera.

2.3 Presupuesto de memoria en Android

Este es el número que decide si el proyecto corre en el teléfono que tengas.

Componente	RAM aprox.
Whisper tiny Q4	~75 MB
Portero Qwen3.5 0.8B Q4	~600 MB
Extractor Qwen3 1.7B Q4	~1,1 GB
KV cache (ctx 2048 × 2 modelos)	~200 MB
Runtime Expo + JS + UI	~300 MB
Total con los tres cargados	~2,3 GB

En un teléfono de 8 GB es viable mantener los tres cargados. En uno de 6 GB conviene descargar Whisper después de transcribir. Por eso el código lleva un PoolDeModelos con política configurable, y no LoadModel sueltos.

Requisitos duros de QVAC en Android, que no se negocian:

- Android **12 o superior**, arquitectura **arm64**
- GPU **Adreno 700+** con Vulkan, u OpenCL
- **Expo ≥ 54**
- **Solo dispositivo físico**. Los emuladores **no funcionan** por limitaciones de llama.cpp. Cada prueba es build e instalar en el teléfono.

Si tu teléfono no cumple, esto no arranca y no hay forma de saberlo hasta probarlo. **Es lo primero que hay que verificar, antes de escribir una línea.**

2.4 Advertencia sobre los nombres de los modelos

Los identificadores exactos del registro de QVAC **hay que verificarlos**, no asumirlos. En el código van como constantes en un solo archivo, y se confirman así antes de nada:

```
import { modelRegistrySearch } from '@qvac/sdk';
console.log(await modelRegistrySearch({ query: 'qwen' }));
```

De ahí salen los nombres reales. No inventes constantes: si el nombre no existe, LoadModel falla en tiempo de ejecución y perdés una hora buscando el error donde no está.

2.5 Cómo se mide esto de verdad

Para poder afirmar cualquier cifra hace falta un conjunto etiquetado y un protocolo. Sin esto, no hay número que reportar.

Conjunto: las 300 notas, etiquetadas a mano con tieneEquipo: sí/no y, cuando sí, con los lotes correctos. Mitad de cada tipo, incluyendo casos difíciles: notas que mencionan un hospital pero ningún equipo, notas con números que no son cantidades ("*llegué a las 3*"), notas con marcas mencionadas de pasada.

Se reportan cinco números, no dos:

Métrica	Qué mide	Por qué importa
Falsos positivos del extractor	inventa filas donde no hay	el problema original
Aciertos del portero	detecta las notas vacías	la solución propuesta
Tasa de fallo conjunto	ambos se equivocan a la vez	si es alta, el portero no aporta
Cobertura del verificador de evidencia	filas fabricadas atrapadas por código	el aporte determinista
Tasa de interrupción	preguntas por cada 100 notas	si es alta, nadie lo usa dos veces

La tercera es la que valida o refuta la arquitectura entera. La quinta es la que decide si el producto se adopta.

Y hasta tenerlas: **en el README y en el video se dice "esperamos", no "logramos".**

PARTE 3 · EL CÓDIGO ANDROID, EXPLICADO

3.1 Qué se reutiliza y qué es nuevo

De QUÓRUM escritorio viaja al teléfono **sin tocar una línea**:

```
core/contracts.ts  core/ids.ts          trust/normalize.ts
trust/similarity.ts trust/entity.ts      trust/reconcile.ts ← el motor
trust/score.ts     policy/engine.ts    context/spotlight.ts
export/philips.ts
```

Eso es posible porque el motor de reconciliación es una función pura sin entrada/salida. Es el pago de una decisión de diseño que tomamos al principio.

Es nuevo o cambia:

```
qvac/pool.ts      gestión de memoria de tres modelos
pipeline/portero.ts  NUEVO
pipeline/extractor.ts extracción con evidencia citada
pipeline/verificar.ts NUEVO – determinista
pipeline/cruzar.ts  NUEVO – enrutador de interrupciones
store/expo-store.ts node:fs → expo-file-system
app/*              UI en React Native
```

3.2 `src/qvac/pool.ts` — tres modelos en un teléfono

El error clásico es llamar a `loadModel` cada vez. Cargar un modelo de 1,1 GB tarda segundos y fragmenta memoria. El pool los mantiene y los descarga bajo presión.

```
import { loadModel, unloadModel, getLoadedModelInfo } from '@qvac/sdk';

/** &#9888; VERIFICAR estos identificadores con modelRegistrySearch antes de usarlos. */
export const MODELOS = {
  asr: 'WHISPER_TINY',
  portero: 'QWEN3_5_0_8B_INST_Q4',
  extractor: 'QWEN3_1_7B_INST_Q4',
} as const;

type Rol = keyof typeof MODELOS;

const cargados = new Map<Rol, string>();

/** ctx_size pequeño a propósito: una nota de campo son 2 o 3 frases.
 * Un contexto grande gasta RAM que en un teléfono no sobra. */
const CONFIG: Record<Rol, Record<string, unknown>> = {
  asr: {},
  portero: { ctx_size: 1024 },
  extractor: { tools: true, ctx_size: 2048 },
};

const TIPO: Record<Rol, string> = {
  asr: 'whisper', portero: 'llm', extractor: 'llm',
};

export async function obtener(rol: Rol): Promise<string> {
  const ya = cargados.get(rol);
  if (ya) return ya;

  const modelId = await loadModel({
    modelSrc: MODELOS[rol],
    modelType: TIPO[rol],
    modelConfig: CONFIG[rol],
  });

  // Aserción del plano de control: en el telefono NADA se delega.
  const info = await getLoadedModelInfo({ modelId }) as { isDelegated?: boolean };
  if (info.isDelegated) {
```

```

    await unloadModel({ modelId });
    throw new Error(`El modelo ${rol} resulto delegado y la politica exige local`);
  }

  cargados.set(rol, modelId);
  return modelId;
}

/** Se llama tras transcribir en dispositivos con poca RAM. */
export async function liberar(rol: Rol): Promise<void> {
  const id = cargados.get(rol);
  if (!id) return;
  await unloadModel({ modelId: id });
  cargados.delete(rol);
}

export async function liberarTodo(): Promise<void> {
  for (const rol of [...cargados.keys()]) await liberar(rol);
}

```

3.3 src/pipeline/precheck.ts — la señal que no cuesta nada

```

import { MODALIDADES, MARCAS_DUMMY } from '../core/contracts';
import { normalizarModalidad } from '../trust/normalize';

const PALABRAS_MODALIDAD = [
  'mr', 'mri', 'resonancia', 'resonador', 'resonadores', 'rm',
  'ct', 'tac', 'tomografo', 'tomografos', 'tomografia', 'scanner', 'escaner',
  'ecografo', 'ecografos', 'ecografia', 'eco', 'ultrasonido', 'ultrasound',
  'rayos', 'xray', 'monitor', 'monitores', 'monitoreo', 'igt',
];

export interface Indicios {
  hayIndicios: boolean;
  modalidades: string[];
  hayNumeros: boolean;
  hayMarca: boolean;
}

/** Señal barata e INDEPENDIENTE de cualquier modelo.
 * Nunca decide sola: solo acompaña al portero para permitir el atajo. */
export function precheck(texto: string): Indicios {
  const t = texto.normalize('NFD').replace(/[\u0300-\u036f]/g, '').toLowerCase();
  const modalidades = PALABRAS_MODALIDAD.filter((p) => new RegExp(`\\b${p}`).test(t));
  const hayNumeros = /\b(\d+|un|una|dos|tres|cuatro|cinco|seis|siete|ocho|nueve|diez)\b/.test(t);
  const hayMarca = MARCAS_DUMMY.some((m) =>
    t.includes(m.toLowerCase().split(' ')[0]));
  return {
    modalidades, hayNumeros, hayMarca,
    hayIndicios: modalidades.length > 0 || hayMarca,
  };
}

```

3.4 src/pipeline/portero.ts — el modelo de 0.8B

Una sola decisión binaria. El prompt es corto a propósito: un modelo de 0.8B con un prompt largo se pierde.

```

import { z } from 'zod';
import { completion } from '@qvac/sdk';
import { obtener } from '../qvac/pool';

const zVeredicto = z.object({
  hayEquipo: z.boolean().describe('true si la nota describe equipos medicos instalados'),
  motivo: z.string().max(120),
});

const TOOL_PORTERO = {

```

```

    name: 'responder',
    description: 'Responde si la nota describe equipos medicos instalados en un cliente.',
    parameters: zVeredicto,
  });

const SISTEMA = `Decides UNA cosa: si la nota describe equipos medicos INSTALADOS
en un hospital o clinica.

hayEquipo = true -> menciona resonadores, tomografos, ecografos, rayos X,
                    monitores u otro equipo medico presente en el sitio.
hayEquipo = false -> solo habla de la visita, de personas, de logistica,
                    o no pudo entrar y no vio nada.

No extraigas datos. Solo responde con la herramienta.`;

export interface Veredicto { hayEquipo: boolean; motivo: string }

export async function portero(nota: string): Promise<Veredicto> {
  const modelId = await obtener('portero');
  const run = completion({
    modelId,
    history: [
      { role: 'system', content: SISTEMA },
      { role: 'user', content: nota },
    ],
    stream: false,
    tools: [TOOL_PORTERO],
    generationParams: { temp: 0, seed: 42, predict: 80 },
  });

  const final = await run.final as {
    contentText?: string;
    toolCalls?: Array<{ name: string; arguments: unknown }>;
  };

  const call = final.toolCalls?.find((c) => c.name === 'responder');
  const p = zVeredicto.safeParse(call?.arguments);

  // Fail-open hacia el extractor: si el portero falla, NO bloqueamos la nota.
  // Un portero roto nunca puede impedir que se capture informacion.
  if (!p.success) return { hayEquipo: true, motivo: 'portero sin respuesta valida' };
  return p.data;
}

```

El **fail-open** de la última línea es deliberado y es una decisión de seguridad. El portero es un filtro de conveniencia; si se rompe, el sistema debe degradar hacia *capturar de más*, nunca hacia *perder información*. Lo contrario —fail-closed— sería correcto para una acción peligrosa, pero aquí la acción peligrosa es **descartar**.

3.5 `src/pipeline/extractor.ts` — con evidencia obligatoria

El cambio respecto a la versión de escritorio es el campo `evidencia`.

```

const zExtraccion = z.object({
  cliente: z.string(),
  ciudad: z.string().optional(),
  pais: z.string().optional(),
  sitio: z.string().optional(),
  notas: z.string().optional(),
  lotes: z.array(z.object({
    modalidad: z.string(),
    cantidad: z.number().int().optional(),
    marca: z.string().optional(),
    modelo: z.string().optional(),
    edadAnios: z.number().optional(),
    // * obligatorio y verificable por codigo
    evidencia: z.string().describe(
      'FRAGMENTO LITERAL Y EXACTO de la nota que justifica esta fila. ' +
      'Copialo tal cual, sin resumir, sin reescribir, sin traducir.'),
  })).min(1),
});

```

```
});
```

Y en el prompt del sistema, la instrucción que lo hace funcionar:

```
- Cada lote DEBE incluir `evidencia`: el trozo textual de la nota que lo respalda, copiado palabra por palabra. Si no puedes copiar un fragmento literal que lo justifique, NO generes ese lote.
```

Esa última frase hace doble trabajo: pide la cita y, de paso, le da al modelo una salida honesta cuando no tiene respaldo.

3.6 `src/pipeline/verificar.ts` — determinista, y es el corazón

```
/** Normalizacion tolerante: minusculas, sin acentos, espacios colapsados.
 * Tolerante a formato, ESTRUCTA en contenido. */
function norm(s: string): string {
  return s.normalize('NFD').replace(/[\u0300-\u036f]/g, '')
    .toLowerCase().replace(/[^a-z0-9\s]/g, ' ')
    .replace(/\s+/g, ' ').trim();
}

export interface Verificacion {
  valida: boolean;
  razon?: string;
}

/** ¿La cita existe realmente en la nota?
 * Sin modelo. Sin umbral. Sin correlacion de errores. */
export function verificarEvidencia(nota: string, evidencia: string): Verificacion {
  const n = norm(nota), e = norm(evidencia);

  if (e.length < 3) return { valida: false, razon: 'evidencia vacia o trivial' };
  if (n.includes(e)) return { valida: true };

  // Tolerancia acotada: el modelo puede omitir una palabra al copiar.
  // Se exige que TODAS las palabras de la cita esten en la nota
  // y que al menos una secuencia de 3 palabras coincida.
  const palabras = e.split(' ');
  if (!palabras.every((p) => n.includes(p))) {
    return { valida: false, razon: 'la cita contiene palabras ausentes en la nota' };
  }
  for (let i = 0; i + 3 <= palabras.length; i++) {
    if (n.includes(palabras.slice(i, i + 3).join(' '))) return { valida: true };
  }
  return { valida: false, razon: 'la cita no aparece como secuencia en la nota' };
}
```

Por qué la tolerancia está acotada así: exigir coincidencia exacta produciría falsos rechazos cada vez que el modelo normaliza una tilde o cambia una coma. Aceptar cualquier parecido dejaría pasar fabricaciones. Exigir que *todas* las palabras estén presentes **y** que haya una secuencia de tres consecutivas es el punto donde una fila inventada casi nunca pasa y una cita real casi siempre pasa.

3.7 `src/pipeline/cruzar.ts` — el enrutador de interrupciones

```
import type { Observacion } from '../core/contracts';
import { precheck } from '../precheck';
import { portero } from '../portero';
import { extraer } from '../extractor';
import { verificarEvidencia } from '../verificar';

export type Resultado =
  | 'ACUERDO' | 'ACUERDO_VACIO'
  | 'POSIBLE_OMISION_PORTERO' | 'POSIBLE_OMISION_EXTRACTOR'
  | 'EVIDENCIA_FABRICADA';

export interface SalidaPipeline {
  resultado: Resultado;
  lotes: Observacion[]; // solo los que tienen evidencia valida
  descartados: Array<{ lote: unknown; razon: string }>; // se MARCAN, no se borran
```

```

pregunta: string | null;          // maximo UNA
traza: {
  hayIndicios: boolean; porteroDijo: boolean; porteroMotivo: string;
  lotesPropuestos: number; lotesValidos: number; atajo: boolean;
  msPortero: number; msExtractor: number;
};
}

export async function procesarNota(
  nota: string, ctx: { observadorId: string; dispositivoId: string; visitadoEn?: string },
): Promise<SalidaPipeline> {

  const ind = precheck(nota);

  const t0 = Date.now();
  const v = await portero(nota);
  const msPortero = Date.now() - t0;

  const trazaBase = {
    hayIndicios: ind.hayIndicios, porteroDijo: v.hayEquipo,
    porteroMotivo: v.motivo, msPortero,
  };

  // — ATAJO: dos senales independientes coinciden en que no hay nada.
  //   La nota se GUARDA igual: "visite y no observe equipo" es un dato.
  if (!v.hayEquipo && !ind.hayIndicios) {
    return {
      resultado: 'ACUERDO_VACIO', lotes: [], descartados: [], pregunta: null,
      traza: { ...trazaBase, lotesPropuestos: 0, lotesValidos: 0,
        atajo: true, msExtractor: 0 },
    };
  }

  const t1 = Date.now();
  const propuestos = await extraer(nota, ctx);
  const msExtractor = Date.now() - t1;

  // — Verificacion determinista de cada cita
  const validos: Observacion[] = [];
  const descartados: Array<{ lote: unknown; razon: string }> = [];
  for (const l of propuestos) {
    const chk = verificarEvidencia(nota, l.evidencia);
    if (chk.valida) validos.push(l);
    else descartados.push({ lote: l, razon: chk.razon! });
  }

  const traza = { ...trazaBase, lotesPropuestos: propuestos.length,
    lotesValidos: validos.length, atajo: false, msExtractor };

  // — Tabla de decision. Maximo UNA pregunta.
  if (descartados.length > 0) {
    return { resultado: 'EVIDENCIA_FABRICADA', lotes: validos, descartados,
      pregunta: `Descarte ${descartados.length} fila(s) sin respaldo en tu nota. ` +
        `¿Quieres revisarlas?`, traza };
  }
  if (v.hayEquipo && validos.length > 0) {
    return { resultado: 'ACUERDO', lotes: validos, descartados: [], pregunta: null, traza };
  }
  if (!v.hayEquipo && validos.length > 0) {
    return { resultado: 'POSIBLE_OMISION_PORTERO', lotes: validos, descartados: [],
      pregunta: 'Detecte estos equipos en tu nota. ¿Es correcto?', traza };
  }
  if (v.hayEquipo && validos.length === 0) {
    return { resultado: 'POSIBLE_OMISION_EXTRACTOR', lotes: [], descartados: [],
      pregunta: 'Creo que mencionaste equipo pero no logre estructurarlo. ' +
        '¿Podes decirlo con el tipo y la cantidad?', traza };
  }
  return { resultado: 'ACUERDO_VACIO', lotes: [], descartados: [], pregunta: null, traza };
}

```

Fijate en tres cosas del código:

1. **descartados nunca se pierde.** Va a la interfaz y al log. *Marcar en vez de borrar*, que era el acierto nº1 del diseño original.
2. **pregunta es un solo string, no un array.** El tipo mismo impone el presupuesto de una interrupción por nota.
3. **traza lleva los milisegundos de cada modelo.** Sin eso no podés medir la tasa de interrupción ni justificar si el portero vale su latencia.

3.8 `src/store/expo-store.ts`

```
import * as FileSystem from 'expo-file-system';
import { zObservacion, type Observacion } from '../core/contracts';

const RUTA = FileSystem.documentDirectory + 'observaciones.jsonl';
let memoria: Observacion[] | null = null;

export async function cargar(): Promise<Observacion[]> {
  if (memoria) return memoria;
  try {
    const txt = await FileSystem.readAsStringAsync(RUTA);
    memoria = txt.trim().split('\n').filter(Boolean)
      .map((l) => zObservacion.parse(JSON.parse(l)));
  } catch { memoria = []; }
  return memoria;
}

export async function agregar(obs: Observacion[]): Promise<number> {
  const actual = await cargar();
  const vistos = new Set(actual.map((o) => o.id));
  const nuevas = obs.filter((o) => !vistos.has(o.id));
  if (!nuevas.length) return 0;
  const previo = await FileSystem.readAsStringAsync(RUTA).catch(() => '');
  await FileSystem.writeAsStringAsync(
    RUTA, previo + nuevas.map((o) => JSON.stringify(o)).join('\n') + '\n';
  actual.push(...nuevas);
  return nuevas.length;
}
```

3.9 Captura de audio con `expo-av`

```
import { Audio } from 'expo-av';
import { transcribe } from '@qvac/sdk';
import { obtener, liberar } from '../qvac/pool';

let grabacion: Audio.Recording | null = null;

export async function iniciarGrabacion(): Promise<void> {
  const permiso = await Audio.requestPermissionsAsync();
  if (!permiso.granted) throw new Error('Permiso de microfono denegado');
  await Audio.setAudioModeAsync({ allowsRecordingIOS: true, playsInSilentModeIOS: true });
  const { recording } = await Audio.Recording.createAsync(
    Audio.RecordingOptionsPresets.HIGH_QUALITY);
  grabacion = recording;
}

export async function detenerYTranscribir(pocaRam = false): Promise<string> {
  if (!grabacion) throw new Error('No hay grabacion activa');
  await grabacion.stopAndUnloadAsync();
  const uri = grabacion.getURI();
  grabacion = null;
  if (!uri) throw new Error('No se obtuvo el audio');

  const modelId = await obtener('asr');
  const r = await transcribe({ modelId, audio: uri }) as { text?: string };

  // En dispositivos ajustados de RAM, liberar el ASR antes de los LLM.
  if (pocaRam) await liberar('asr');
  return r.text ?? '';
}
```

Nunca se usa reconocimiento de voz del sistema operativo ni del navegador: eso envía el audio a servidores del proveedor y **es inferencia en la nube**. Descalifica la entrega completa.

PARTE 4 · PROMPT MAESTRO PARA EL AGENTE DE CODIFICACIÓN

Pegar íntegro como primer mensaje. Las secciones marcadas como restricciones duras no se negocian.

```
Actúas como tech lead y arquitecto de una app Android que compite en el
Decentralized AI Hackathon (reto corporativo de Philips: Customer Installed
Base Intelligence). El proyecto se llama QUÓRUM. Lema: "La verdad tiene quórum".

=== RESTRICCIONES DURAS – violarlas descalifica la entrega ===

1. TODA la inferencia corre en el dispositivo mediante QVAC (@qvac/sdk).
   Ninguna llamada a OpenAI, Anthropic, Gemini, Groq, Together, Replicate
   ni ningún endpoint de inferencia remoto, en ningún punto del código.
2. PROHIBIDO el reconocimiento de voz del sistema operativo o del navegador
   (SpeechRecognition, Web Speech API). Envía audio a la nube. La
   transcripción se hace con whisper.cpp vía QVAC, on-device.
3. PROHIBIDO @qvac/ai-sdk-provider y el Vercel AI SDK (política de la
   organización). Usar @qvac/sdk puro.
4. PROHIBIDO Vercel para cualquier cosa.
5. Cero credenciales, API keys o tokens en el código.
6. Solo vocabulario de marcas FICTICIO: NovaMed, Aurelia Health, BluePeak
   Medical, Orion Imaging, HelixCare, Zenith MedTech. NUNCA marcas reales
   de equipo médico, ni Philips ni la competencia.
7. Instalar dependencias con --ignore-scripts. Antes de añadir cualquier
   dependencia nueva, justificá por qué el beneficio supera el riesgo de
   cadena de suministro.

=== PLATAFORMA ===

Expo ≥54, React Native, TypeScript estricto, Android 12+ arm64, dispositivo
FÍSICO (los emuladores no funcionan por limitaciones de llama.cpp).
Node ≥22.17 para el entorno de desarrollo.

=== PASO 0 – ANTES DE ESCRIBIR CÓDIGO ===

Ejecutá y mostrame la salida de:
npx --package "@qvac/cli" qvac doctor
node -e "import('@qvac/sdk').then(m=>m.modelRegistrySearch({query:'qwen'})).then(console.log))"
node -e "import('@qvac/sdk').then(m=>m.modelRegistrySearch({query:'whisper'})).then(console.log))"

Necesito los identificadores REALES del registro para el portero (~0.8B),
el extractor (~1.7B) y whisper. NO inventes constantes de modelo: si el
nombre no existe, loadModel falla en tiempo de ejecución.

Si qvac doctor no detecta Vulkan ni OpenCL, PARÁ y avisame antes de seguir.

=== ARQUITECTURA – pipeline de tres modelos ===

audio → whisper (ASR) → texto → precheck determinista → PORTERO 0.8B
  → [atajo si ambos dicen que no hay nada]
  → EXTRACTOR 1.7B con evidencia citada
  → VERIFICADOR DE EVIDENCIA (código, sin modelo)
  → cruzar() enrutador de interrupciones
  → BORRADOR → confirmación humana → store → motor de quórum

PRINCIPIO RECTOR, no negociable:
  El LLM entiende. El código decide. El humano confirma.

- La salida de todo modelo es INPUT HOSTIL: se valida con Zod o se rechaza.
- Ningún modelo puede cambiar un estado de confianza.
- Nada se persiste sin confirmación humana explícita.
- NADIE BORRA: lo que se rechaza se MARCA y queda visible.
- Máximo UNA pregunta por nota. Si el usuario no responde, la nota se
  guarda con estado pendiente-de-revisión. Nunca se descarta.

EL CRUCE NO DECIDE QUÉ ES VERDAD. Decide si el usuario ve un borrador limpio
```

o una pregunta. Es un enrutador de interrupciones. El árbitro es el humano, y después el quórum entre observadores independientes.

Tabla de decisión (implementar exactamente así):

portero	lotes	evidencia	resultado	acción
----- ----- ----- ----- -----				
sí	N>0	toda válida	ACUERDO	borrador limpio
no	0	-	ACUERDO_VACIO	guarda nota vacía
no	N>0	toda válida	POSIBLE_OMISION_PORTERO	1 pregunta
sí	0	-	POSIBLE_OMISION_EXTRACTOR	1 pregunta
-	N>0	alguna inválida	EVIDENCIA_FABRICADA	marca + 1 pregunta

EVIDENCIA CITADA – es la pieza central:

Cada lote que produce el extractor DEBE traer el fragmento LITERAL de la nota que lo justifica. Un verificador determinista comprueba que esa cita existe realmente en el texto. Si no existe, la fila está fabricada y no se guarda. Esto NO depende de que dos modelos fallen distinto: es comparación de cadenas.

== MODELO DE DATOS ==

Observación (testimonio inmutable): id, sesionId, observadorId, dispositivoId, visitadoEn (fecha de la VISITA, distinta de capturadaEn), capturadaEn, fuente (voz|texto), naturaleza (Directo|Referido|Estimado|Desconocido – mapea al Status de Philips: Confirmed|Reported|Estimated|Unknown), origen (local|peer), cliente {nombre, ciudad?, pais?, sitio?}, lote {modalidad, marca?, modelo?, cantidad?, edadAnios?}, evidencia (cita literal), hedging (booleano, detectado por CÓDIGO sobre el texto, no por el modelo), notas?, seguimiento[], provenance {hash, modeloSha256?, delegado}.

Un LOTE = N unidades que comparten edad. "Tres MR, dos viejos y uno nuevo" produce DOS lotes de la misma modalidad (2 y 1). El total del grupo es la SUMA de los lotes de una sesión. Sin esto se genera un conflicto falso 2-vs-1.

Motor de quórum, 7 reglas deterministas (función PURA, sin I/O, testeable):

- RD-0 sin datos → "Sin datos". No se rellena con supuestos.
- RD-1 un solo testigo nunca produce quórum.
- RD-2 testigos que discrepan → "Sin quórum". NO se promedia jamás.
- RD-3 un observador cuenta UNA vez (su testimonio más reciente).
- RD-4 confianza POR CAMPO: discrepar en edad no degrada marca ni cantidad.
- RD-5 si todos hedgearon, el techo es "Reportado".
- RD-6 la frescura se mide desde visitadoEn, no desde capturadaEn.
- RD-7 solo un testimonio Directo y asertivo puede otorgar quórum.

== ORDEN DE CONSTRUCCIÓN ==

- Contratos y tipos (Zod). Congelalos: no cambian después.
- Pool de modelos con presupuesto de RAM y aserción de que isDelegated=false.
- Hola mundo: cargar el portero y obtener un booleano en el TELÉFONO.
Si esto no funciona, parate y avisame. No sigas depurando a ciegas.
- Precheck determinista + portero + tests.
- Extractor con evidencia + verificador determinista + tests.
- cruzar() con la tabla de decisión + tests de los 5 resultados.
- Motor de quórum: las 7 reglas con un test cada una.
- Store con expo-file-system.
- UI: Capturar (con revisión antes de guardar) y Cliente 360 con estado POR CAMPO.
- Audio con expo-av.

== CÓMO TRABAJÁS ==

- Antes de cada bloque, decime en una línea qué vas a construir y por qué.
- Contrato y test primero, implementación después.
- Después de cada pieza: ejecutar, probar, verificar, corregir, documentar.
No asumas que algo funciona porque el código "se ve bien".
- Si algo del diseño choca con las restricciones duras, PARÁ y avisame.
Las restricciones ganan siempre.
- No inventes capacidades de QVAC, APIs, benchmarks ni métricas. Si no lo sabés, decilo y verificalo contra los .d.ts de la versión instalada.
- Preferí terminar tres cosas a empezar seis.

- Registrá en cada nota: qué dijo el portero, cuántos lotes propuso el extractor, cuántos pasaron la verificación, y los milisegundos de cada modelo. Sin esa traza no se puede medir la tasa de interrupción.

Empezá por el PASO 0 y mostrame la salida antes de escribir código.

PARTE 5 · LO QUE HAY QUE DECIDIR ANTES DE ARRANCAR

1. **¿Tu teléfono cumple?** Android 12+, arm64, Adreno 700+, 8 GB de RAM idealmente. Si no, esto no corre y no hay forma de saberlo sin probarlo. Es lo primero.
2. **¿De dónde salieron el 93 % y el 95 %?** Si son estimaciones, hay que quitarlas del material o marcarlas como hipótesis. Si son medidas, hay que documentar cómo.
3. **Las 300 notas ciegas.** ¿Existen y están etiquetadas? Sin ese conjunto no se puede reportar ninguna de las cinco métricas, y sin la tasa de fallo conjunto no se puede afirmar que el portero aporte algo.
4. **El costo real de la app Android** sigue siendo de 13 a 19 horas sobre las 48, con el sync P2P entre teléfonos como incógnita sin fondo. Mi recomendación anterior sigue en pie: si el objetivo es ganar el hackathon, escritorio primero y teléfono como tercer peer opcional después del freeze. Si el objetivo es que esto viva en el bolsillo de un ingeniero de servicio de verdad, entonces Android es el camino correcto y este documento es el plan.

El pipeline de dos modelos es bueno. Con la evidencia citada, es mucho mejor — porque deja de depender de una suposición sobre correlación de errores que todavía nadie midió.